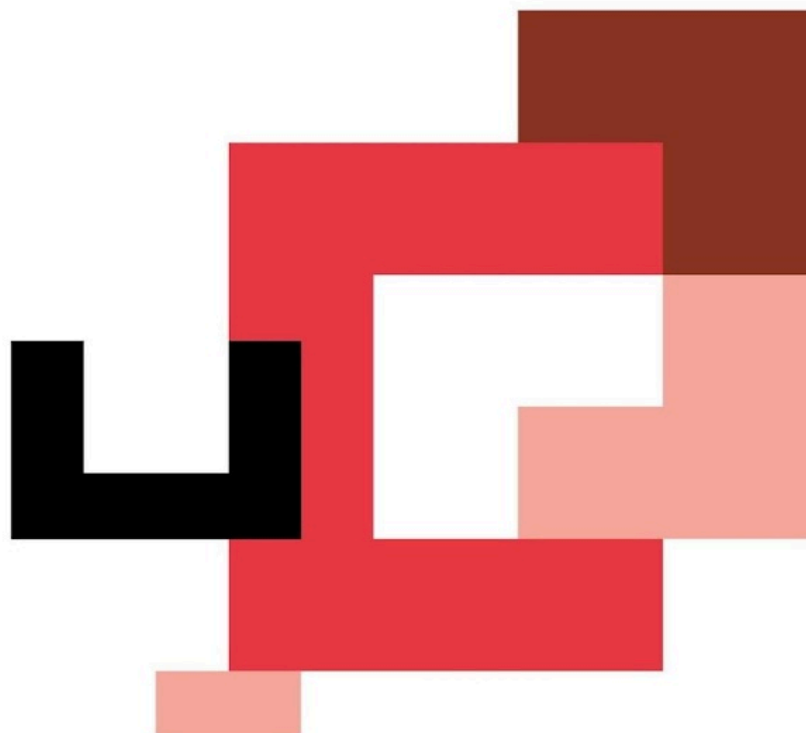




FULL STACK DEVELOPER + IA

Módulo 2 | Feature 3

SQL, Prisma, Supabase



Objetivo de la feature

En esta feature vamos a dar un paso fundamental: **pasar de datos en memoria a una base de datos real**.

Hasta ahora nuestra API funcionaba con datos temporales. En cuanto el servidor se reiniciaba, toda la información desaparecía. A partir de este sprint vamos a persistir datos utilizando **PostgreSQL en Supabase** y **Prisma ORM**.

Que vamos a aprender:

- Cómo conectar una API de **Node.js + Express** a una base de datos real.
- Cómo usar **Prisma ORM** para interactuar con PostgreSQL.
- Cómo definir el **modelo de datos** en `schema.prisma`.
- Cómo usar **variables de entorno (.env)** para gestionar credenciales.
- Cómo centralizar el **manejo de errores en Express** con middlewares.

El objetivo es entender **cómo se conecta un backend profesional a una base de datos relacional** y cómo se organiza el acceso a datos dentro de la arquitectura del proyecto.

Requisitos

Tech / configuración

- Node.js **18+**
- Express
- Prisma ORM
- Supabase (PostgreSQL)
- Proyecto con **ES Modules**

Configuración de la Base de Datos (Supabase)

1. Crea una cuenta en **Supabase**.

<https://supabase.com>

2. Crea un nuevo proyecto.
3. Ve a:

None

Project Settings → **Database**

4.

Copia la **Connection String**.

Debes usar:

- **Transaction mode (puerto 6543)** para **DATABASE_URL**
- **Session mode (puerto 5432)** para **DIRECT_URL** (opcional para migraciones)

Variables de Entorno

Crea un archivo **.env** en la raíz del proyecto:

```
None
DATABASE_URL="postgresql://postgres:[USER]:[PASSWORD]@[HOST]:6543/postgres?pgbouncer=true"

DIRECT_URL="postgresql://postgres:[USER]:[PASSWORD]@[HOST]:5432/postgres"

PORT=3000
```

Importante

El archivo **.env** no debe subirse a GitHub.

Estructura del Proyecto

```
None

src/
├── config/
│   └── prismaClient.js
├── controllers/
├── services/
└── routes/
```

```
├── middlewares/  
│   ├── notFound.js  
│   ├── errorHandler.js  
│   └── validateProduct.js  
├── app.js  
└── server.js  
  
prisma/  
└── schema.prisma  
  
.env
```

Explicación de cada parte

prisma/schema.prisma

Aquí definimos **los modelos de la base de datos**.

Los modelos viven en:

```
None  
prisma/schema.prisma
```

Cuando ejecutamos:

```
None  
npx prisma generate
```

Prisma genera automáticamente un **cliente con autocompletado y validación de tipos**.

prismaClient.js

Este archivo crea una instancia única de Prisma para toda la aplicación.

Ejemplo conceptual:

```
None
const prisma = new PrismaClient()
```

Todos los servicios usarán esta instancia.

Services

Los servicios ahora **ya no trabajan con arrays en memoria**.

En su lugar usan Prisma para acceder a la base de datos.

Ejemplo conceptual:

```
None
prisma.product.findMany()
```

Controllers

Los controladores:

- Validan datos
 - Llaman a los servicios
 - Manejan errores
 - Devuelven respuestas HTTP
-

Middlewares

En este sprint usamos tres middlewares:



Middleware	Función
notFound	Maneja rutas inexistentes
errorHandler	Maneja errores globales
validateProduct	Valida body de productos

Modelos Prisma

Product

Campo	Tipo	Descripción
id	String (cuid)	ID autogenerado
name	String	Nombre del producto
description	String?	Descripción opcional
price	Float	Precio
stock	Int	Stock
imageUrl	String?	URL de imagen

createdAt	DateTime	Fecha creación
-----------	----------	----------------

updatedAt	DateTime	Fecha actualización
-----------	----------	---------------------

Endpoints

Método	Ruta	Descripción
GET	/health	Estado del servidor
GET	/api/products	Listar productos
GET	/api/products/:id	Obtener producto
POST	/api/products	Crear producto
PUT	/api/products/:id	Actualizar producto
DELETE	/api/products/:id	Eliminar producto

Flujo de errores en Express

En este sprint implementamos el patrón:



None

```
try {  
  
  ...  
  
} catch(err) {  
  next(err)  
  
}
```

¿Qué hace `next(err)`?

Le dice a Express que **salte los middlewares normales** y busque un **Error Handler**.

Express identifica el Error Handler porque tiene **4 parámetros**:

None

```
(err, req, res, next)
```

El middleware `errorHandler.js` centraliza la respuesta.

Uso de la IA en este sprint

La IA puede ayudarte a entender conceptos o revisar código, pero **no debe escribir todo el sprint por ti**.

Puedes usarla para:

- Revisar el esquema Prisma
 - Detectar validaciones faltantes
 - Revisar queries Prisma
 - Debug de conexión con Supabase
-

Prompts útiles

Revisión del esquema Prisma

None

Estoy modelando una tabla Product en Prisma.

Tiene campos id, name, description, price, stock, imageUrl.

Revisa si el modelo es correcto y qué validaciones debería tener.

Debug de conexión

None

Estoy conectando Prisma con Supabase y obtengo este error.

Te paso DATABASE_URL y el mensaje de error.

Dime las 3 causas más probables.

Revisión de servicios

None

Estoy usando Prisma en mis servicios Express.

¿Es correcto que el controlador solo llame al servicio y que toda la lógica de base de datos esté en services?

Pistas

1. Prisma usa el esquema como fuente de verdad

Si cambias el modelo debes ejecutar:



None

```
npx prisma generate
```

o

None

```
npx prisma db push
```

2. Prisma devuelve Promises

Todos los métodos son asíncronos.

Debes usar:

None

```
await prisma.product.findMany()
```

3. Centraliza errores

No envíes errores desde todos los controladores.

Usa:

None

```
next(err)
```

y deja que el middleware `errorHandler` responda.

Importante que tienen que tener en cuenta

1. No subir `.env`

Nunca subir credenciales al repositorio.

2. Prisma no usa modelos JS

Los modelos viven en:

None
`prisma/schema.prisma`

3. Separación de capas

La arquitectura sigue siendo:

None
`routes → controllers → services → prisma`

4. Datos persistentes

Ahora los datos **sí se guardan en la base de datos**.

Si reinicias el servidor **no se pierden**.

Checks rápidos (autoevaluación)

El sprint está correcto si:

- `npm start` levanta el servidor
 - Prisma conecta con Supabase
 - `GET /api/products` devuelve datos SQL
 - `POST /api/products` crea registros en la BBDD
 - `PUT /api/products/:id` actualiza registros
 - `DELETE /api/products/:id` elimina registros
 - Los errores se manejan con `errorHandler`
-

Ejemplos cURL

Listar productos

Shell

```
curl http://localhost:3000/api/products
```

Crear producto

Shell

```
curl -X POST http://localhost:3000/api/products \
-H "Content-Type: application/json" \
-d '{"name":"Gorra
Deportiva", "price":14.99, "stock":100, "description":"Gorra
ajustable"}'
```

Actualizar producto

Shell

```
curl -X PUT http://localhost:3000/api/products/<ID> \
-H "Content-Type: application/json" \
-d '{"price":12.99}'
```

Eliminar producto

Shell

```
curl -X DELETE http://localhost:3000/api/products/<ID>
```

Notas

- `.env` no debe subirse al repositorio.